UML Diagrams

Insurance Management System

Software Architecture Documentation

- ***Eada Jafrash***

- Diagrams included: Use Case · Class · Sequence · Activity · Component · Package/Module

1. Use Case Diagram

The use case diagram shows the five system actors — Admin, Customer, Surveyor, Underwriter, and Public User — and the nine primary use cases they interact with inside the Insurance Management System boundary.

Actors on the left (Admin, Customer) primarily initiate actions; actors on the right (Surveyor, Underwriter, Public User) respond to or review those actions. All actors share the Login use case.

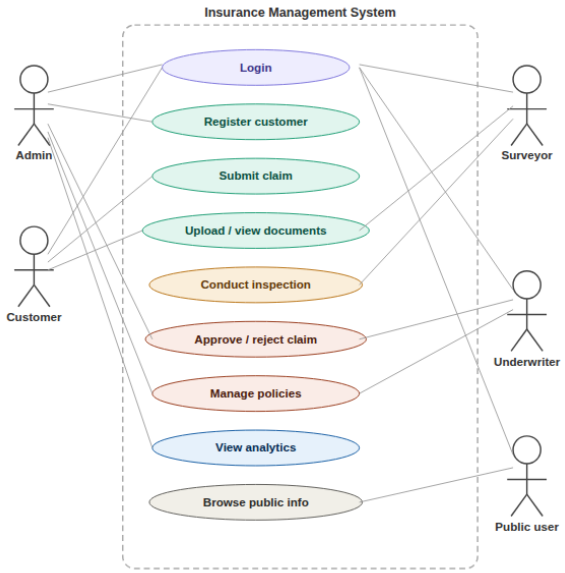
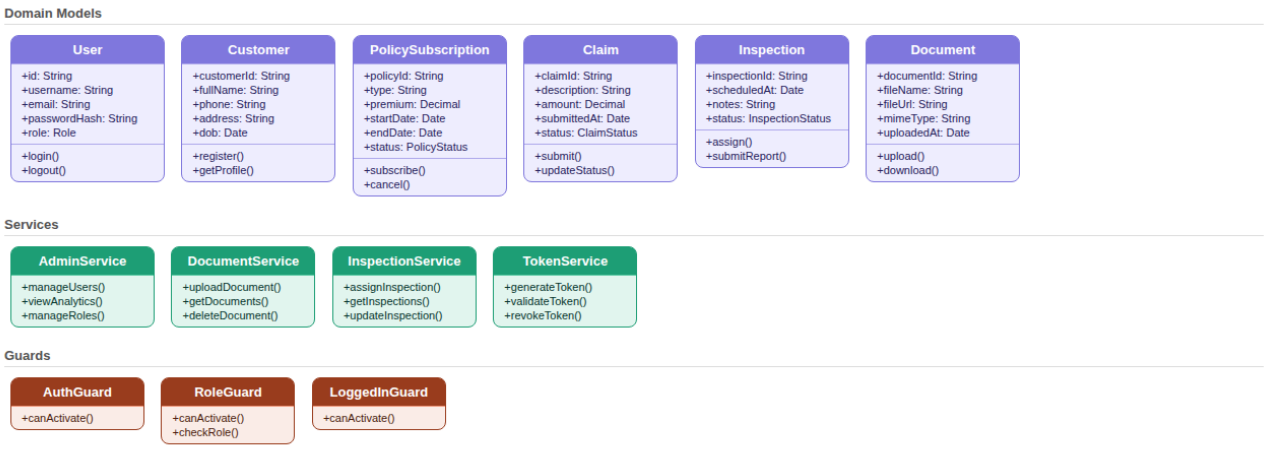


Figure 1: Use Case Diagram — Insurance Management System

2. Class Diagram

The class diagram defines the core domain model, the Angular services that operate on those models, and the route guards that protect navigation. Key relationships include Customer-to-Claim (1 to many), Claim-to-Inspection (1 to 0..1), and Claim/Policy-to-Document (many to many).

Services use dependency relationships (dashed arrows) to the models they manage. Guards depend on TokenService or User for authentication and authorisation checks.



Relationships

Source	Relationship	Target	Multiplicity
User	→ is-a	Customer	1 to 1
Customer	→ holds	PolicySubscription	1 to *
Customer	→ submits	Claim	1 to *
Claim	→ triggers	Inspection	1 to 0..1
Claim	→ has	Document	* to *
PolicySubscription	→ has	Document	* to *
AdminService	..> manages	User	dependency
DocumentService	..> manages	Document	dependency
InspectionService	..> manages	Inspection	dependency
TokenService	..> authenticates	User	dependency
AuthGuard	..> uses	TokenService	dependency
RoleGuard	..> checks	User	dependency
LoggedInGuard	..> uses	TokenService	dependency

Figure 2: Class Diagram — Domain Models, Services, and Guards

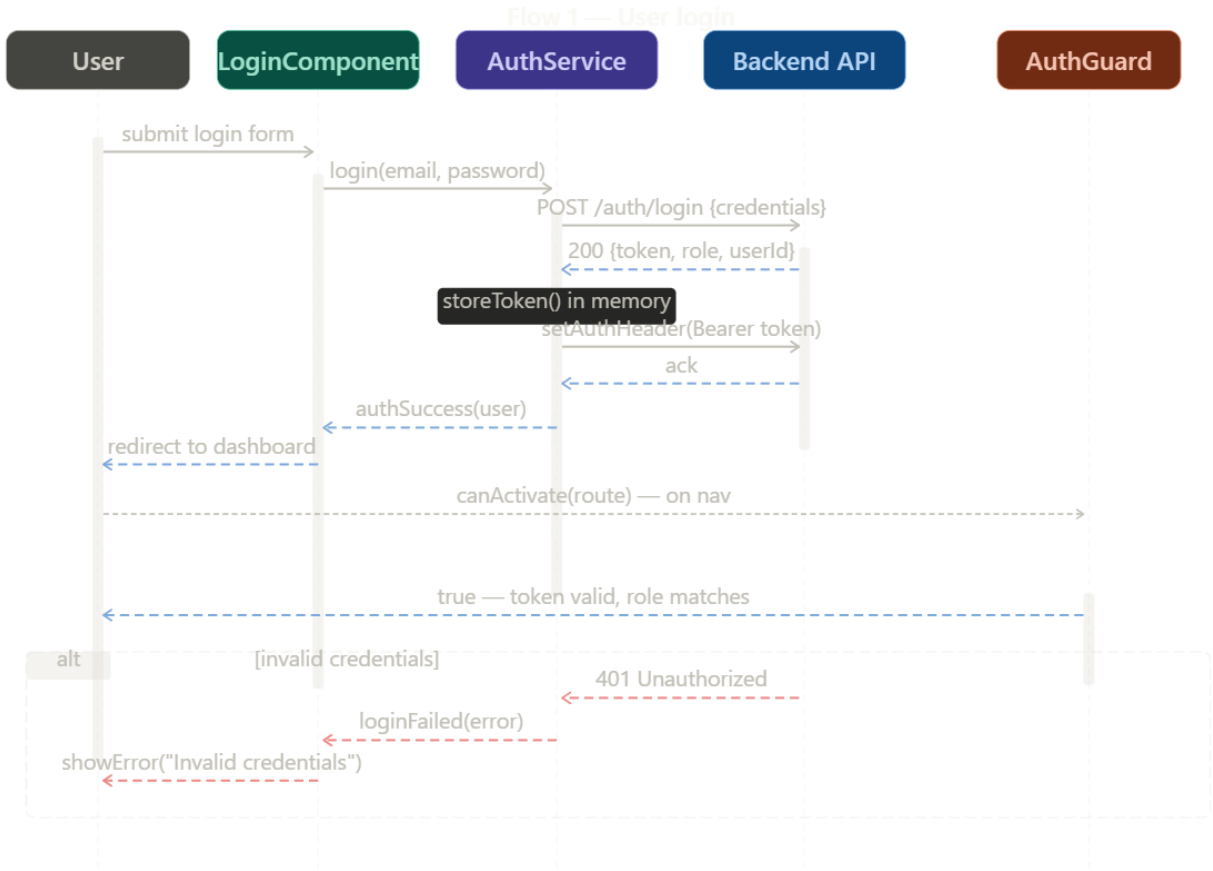
3. Sequence Diagrams

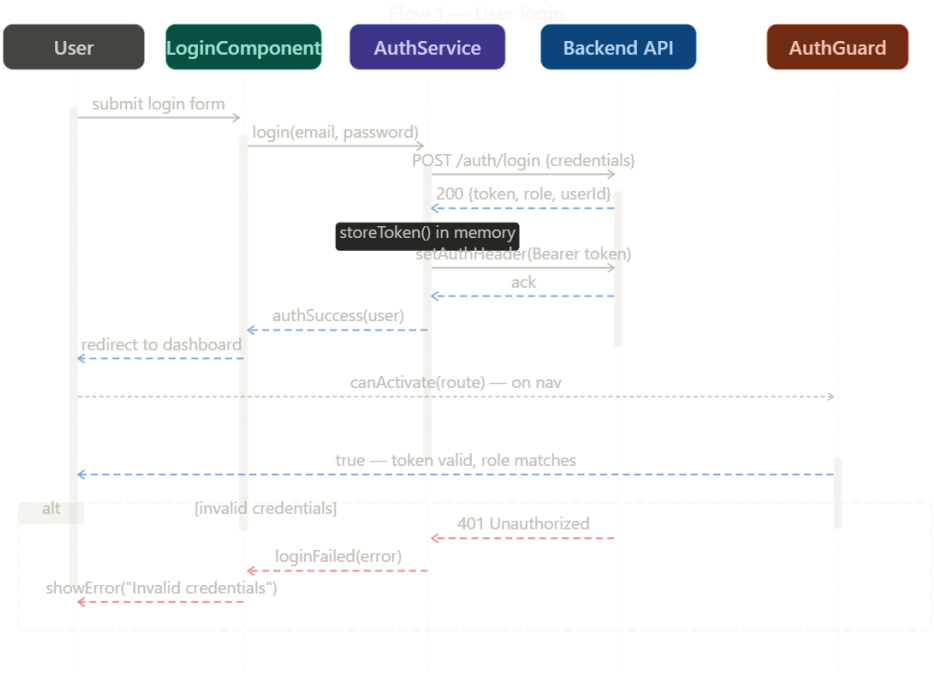
Five key interaction flows are documented below. Each diagram traces a user action through the Angular component layer, service layer, and backend REST API, showing both the outgoing request and the response path.

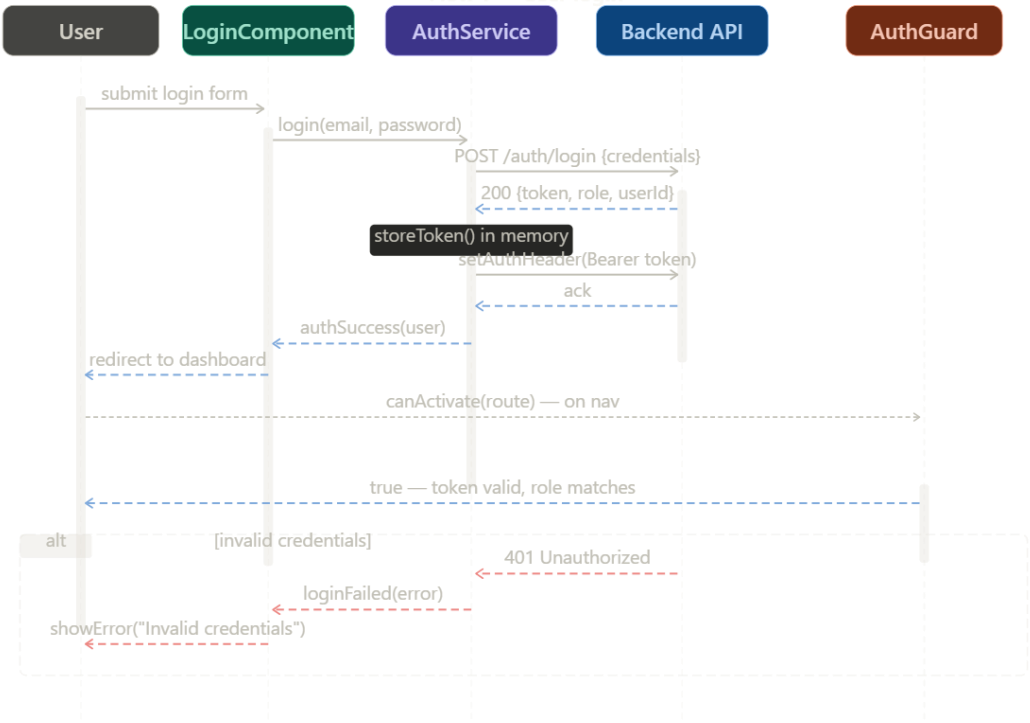
Flows documented:

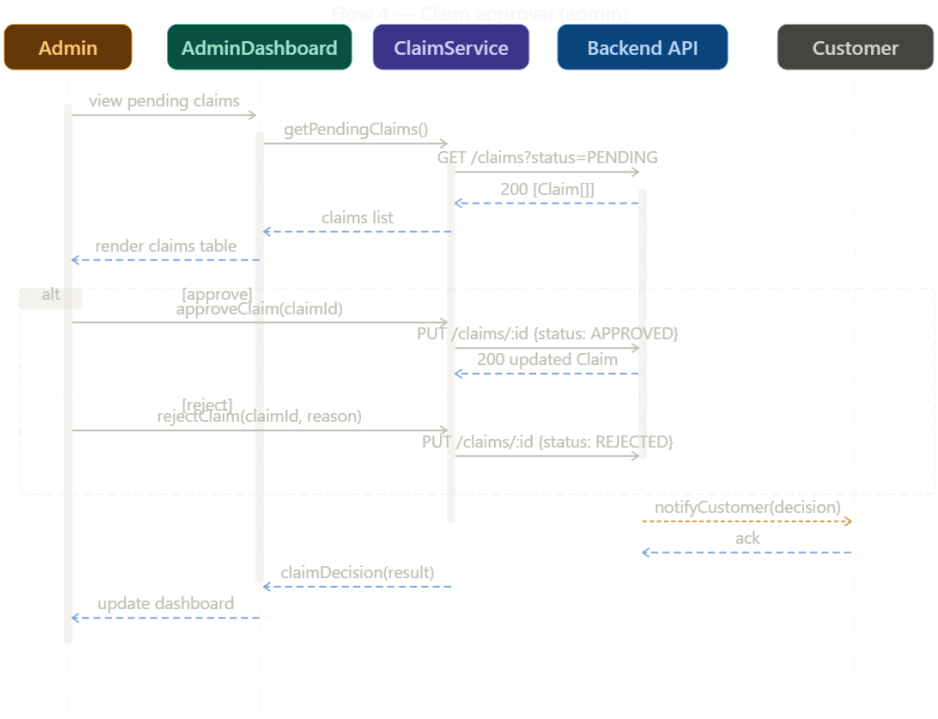
1. Login
2. Claim Submission
3. Policy Approval
4. Inspection Assignment
5. Document Upload

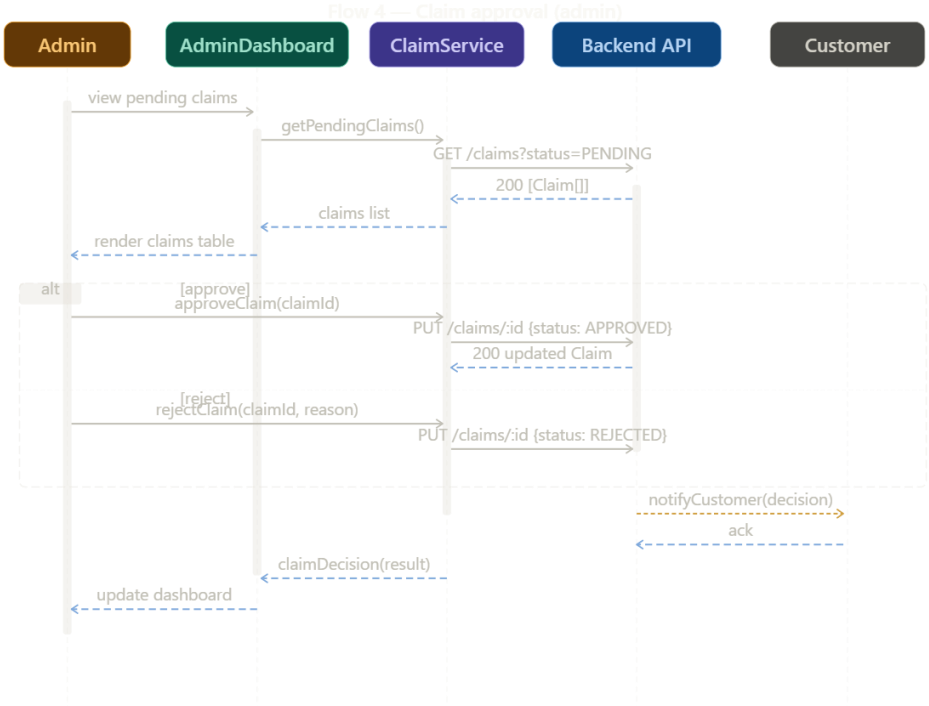
Figure 3: Sequence Diagrams — Five Interaction Flows




User
LoginComponent
AuthService
Backend API
AuthGuard
submit login form
login(email, password)
POST /auth/login {credentials}
200 {token, role, userId}
storeToken() in memory
setAuthHeader(Bearer token)
ack
authSuccess(user)
redirect to dashboard
canActivate(route) — on nav
true — token valid, role matches
alt
[invalid credentials]
401 Unauthorized
loginFailed(error)
showError("Invalid credentials")


User
LoginComponent
AuthService
Backend API
AuthGuard
submit login form
login(email, password)
POST /auth/login {credentials}
200 {token, role, userId}
storeToken() in memory
setAuthHeader(Bearer token)
ack
authSuccess(user)
redirect to dashboard
canActivate(route) — on nav
true — token valid, role matches
alt
[invalid credentials]
401 Unauthorized
loginFailed(error)
showError("Invalid credentials")


Admin
AdminDashboard
ClaimService
Backend API
Customer
view pending claims
getPendingClaims()
GET /claims?status=PENDING
200 [Claim[]]
claims list
render claims table
alt
[approve]
approveClaim(claimId)
PUT /claims/:id {status: APPROVED}
200 updated Claim
[reject]
rejectClaim(claimId, reason)
PUT /claims/:id {status: REJECTED}
notifyCustomer(decision)
ack
claimDecision(result)
update dashboard


Admin
AdminDashboard
ClaimService
Backend API
Customer
view pending claims
getPendingClaims()
GET /claims?status=PENDING
200 [Claim[]]
claims list
render claims table
alt
[approve]
approveClaim(claimId)
PUT /claims/:id {status: APPROVED}
200 updated Claim
[reject]
rejectClaim(claimId, reason)
PUT /claims/:id {status: REJECTED}
notifyCustomer(decision)
ack
claimDecision(result)
update dashboard

4. Activity Diagrams

Activity diagrams model the procedural workflows of the three main business processes. Decision diamonds represent branching points; dashed return arrows show retry loops (e.g. invalid payment returns to the form).

4.1 Claim Processing

Covers the full lifecycle from claim submission through optional inspection assignment, approval/rejection decision, and final settlement or notification.

4.2 Policy Subscription

Shows the customer journey from browsing plans through payment validation to policy activation and confirmation email dispatch.

4.3 Customer Onboarding

Covers registration form submission, data validation loop, account creation, email verification, and admin activation.

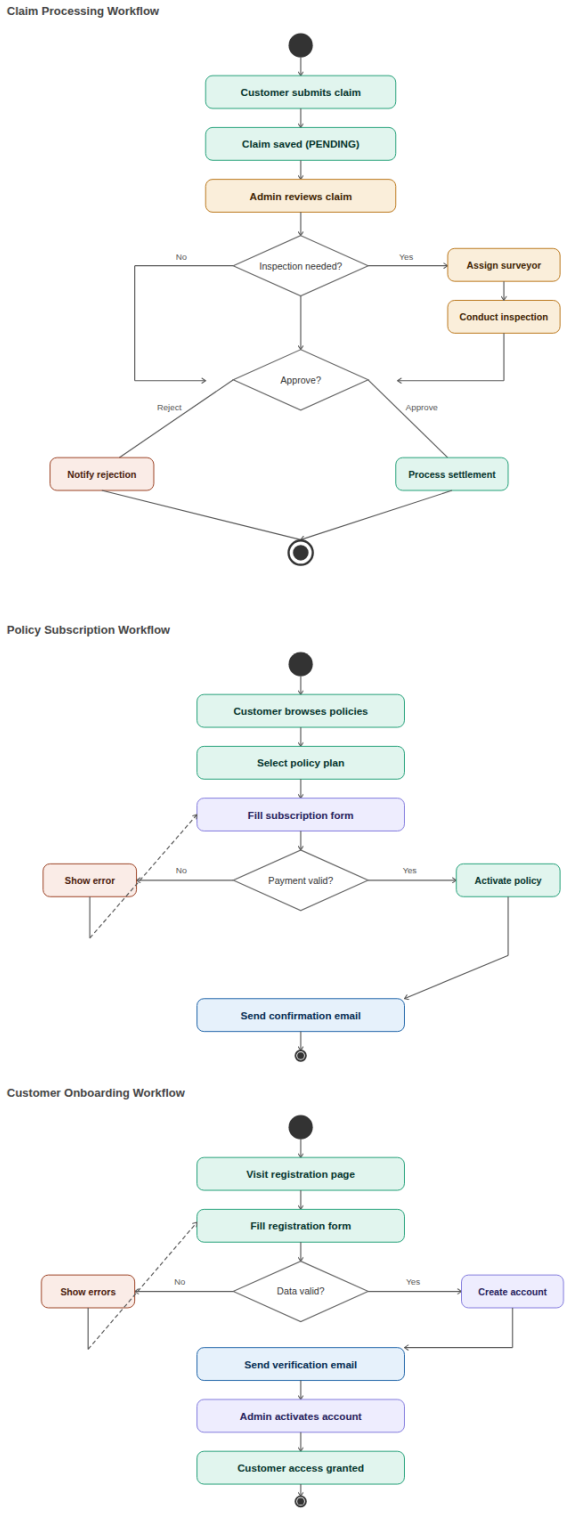


Figure 4: Activity Diagrams — Claim Processing, Policy Subscription, Customer Onboarding

5. Component Diagram

The component diagram illustrates the structural decomposition of the full-stack application. The Angular frontend is split into three layers — Feature Modules, Core, and Shared — each contained within the main system boundary. The backend REST API is shown as a separate component with eight named endpoints, connected to a PostgreSQL database.

Blue dashed arrows represent HTTP service calls from the Angular Core services to backend endpoints. Gray dashed arrows show Angular module import dependencies.

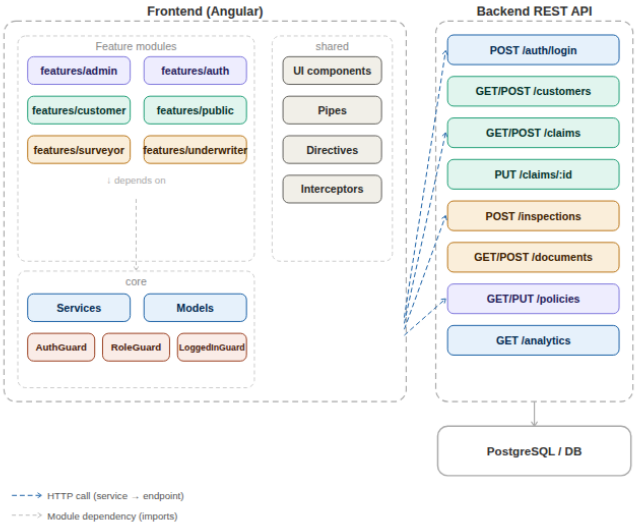


Figure 5: Component Diagram — Frontend, Backend, and Database

6. Package / Module Diagram

The package diagram maps the Angular module architecture. AppModule is the root and imports all six feature modules. Each feature module depends on both CoreModule (services, models, guards) and SharedModule (UI components, pipes, directives, interceptors). Both CoreModule and SharedModule make outbound HTTP calls to the Backend REST API.

This layered dependency structure enforces separation of concerns: business logic lives in core, presentational primitives live in shared, and feature modules contain only feature-specific routing and view components.

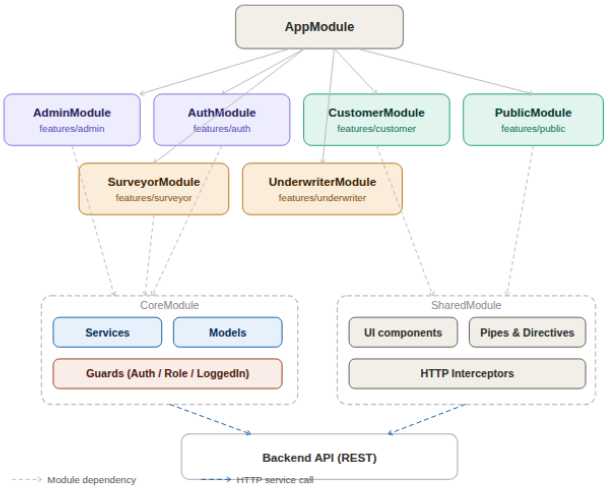


Figure 6: Package / Module Diagram — Angular Module Dependency Graph